Documentação de Arquitetura do Sistema

Sistema de Gestão da Secretaria Escolar DF

1. INTRODUÇÃO

1.1 Finalidade

Este documento descreve a arquitetura do sistema Secretaria DF, detalhando sua estrutura, componentes, interações e decisões técnicas.

O objetivo é fornecer uma visão clara da arquitetura do sistema para desenvolvedores e stakeholders, facilitando manutenção, evolução e entendimento do sistema.

1.2 Escopo

O sistema Secretaria DF é uma aplicação web para gestão de informações administrativas e acadêmicas, incluindo usuários, alunos e registros institucionais.

O acesso é feito via navegador, com processamento no backend e persistência em banco de dados.

1.3 Definições

- MVC: padrão de separação entre Model, View e Controller
- ORM: mapeamento objeto-relacional
- REST: comunicação entre cliente e servidor
- CRUD: operações de criação, leitura, atualização e exclusão

---

1.4 Referências

- Modelo 4+1 de Kruchten
- Boas práticas de Engenharia de Software
- Documentação do projeto (GitHub)

---

2. REPRESENTAÇÃO ARQUITETURAL

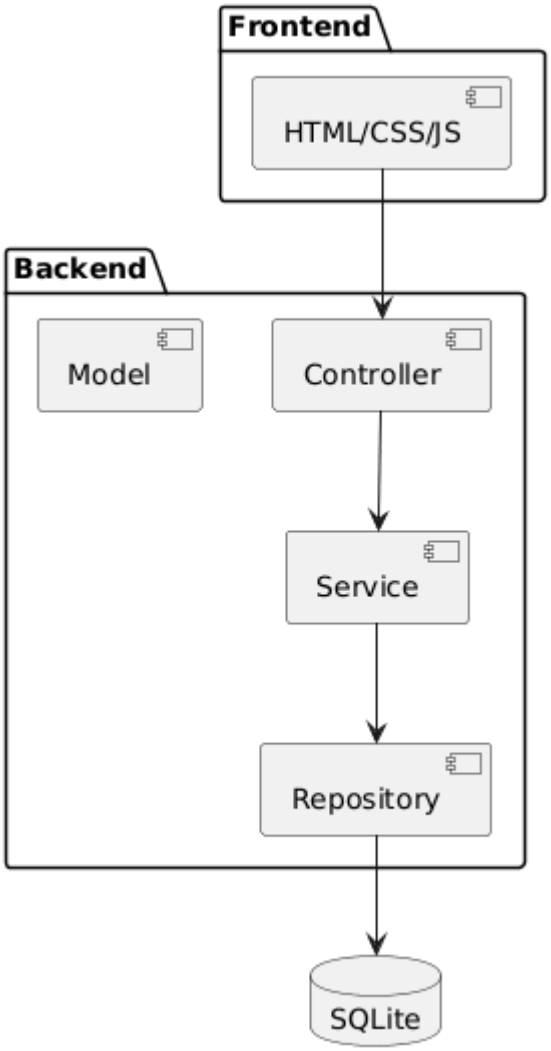
O sistema segue o modelo 4+1 com as seguintes visões:

- Visão lógica
- Visão de processos
- Visão de implementação
- Visão de implantação
- Visão de casos de uso

Estilo arquitetural

- Arquitetura em camadas
- Modelo cliente-servidor
- MVC

DIAGRAMA DE ARQUITETURA GERAL



3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

Requisito	Solução adotada
Linguagem	Python
Framework	Flask ou Django
Banco de Dados	SQLite
Persistência	ORM (ex: SQLAlchemy)
Segurança	Hash de senhas + autenticação
Plataforma	Aplicação Web

Frontend	HTML, CSS, JavaScript
Arquitetura	MVC + Camadas

---

4. VISÃO DE CASOS DE USO

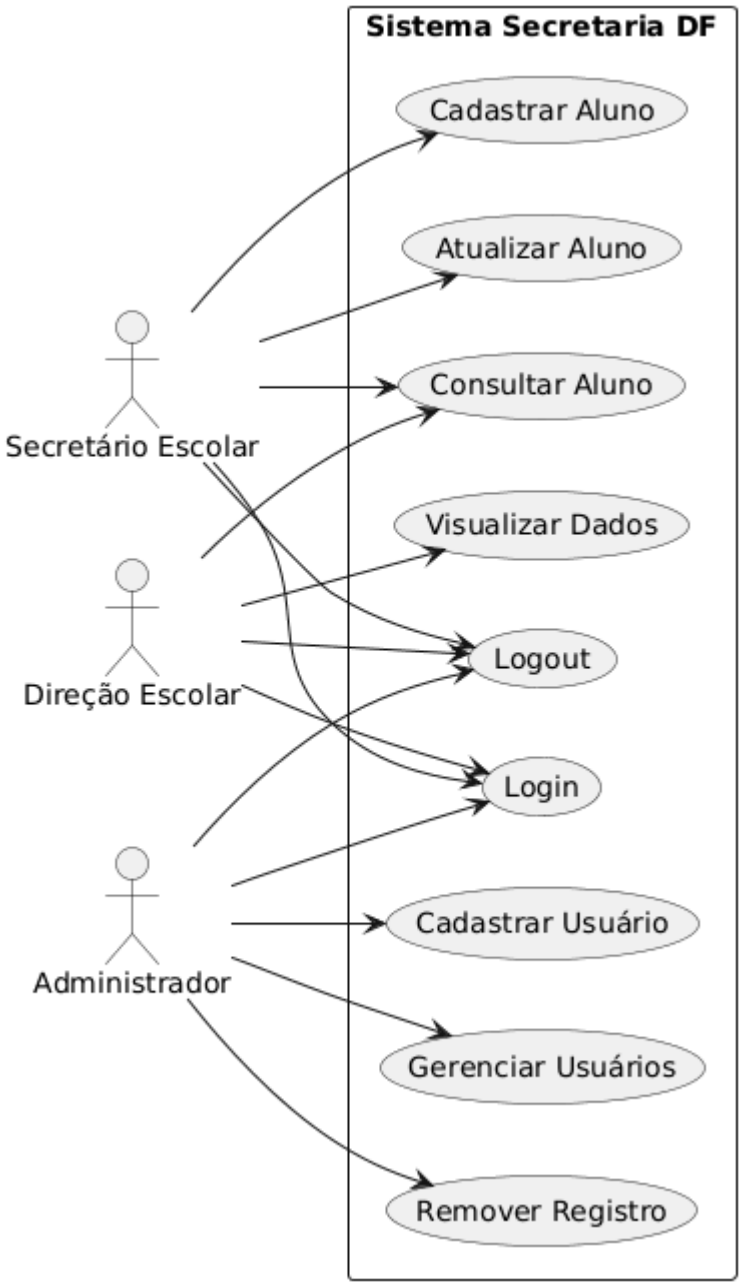
Atores

- Secretário escolar
- Administrador
- Direção escolar

Casos de uso

- Login
- Cadastro de usuário
- Cadastro de aluno
- Consulta de aluno
- Atualização de aluno
- Remoção de registro
- Gerenciamento de usuários
- Visualização de dados administrativos
- Logout

DIAGRAMA DE CASOS DE USO

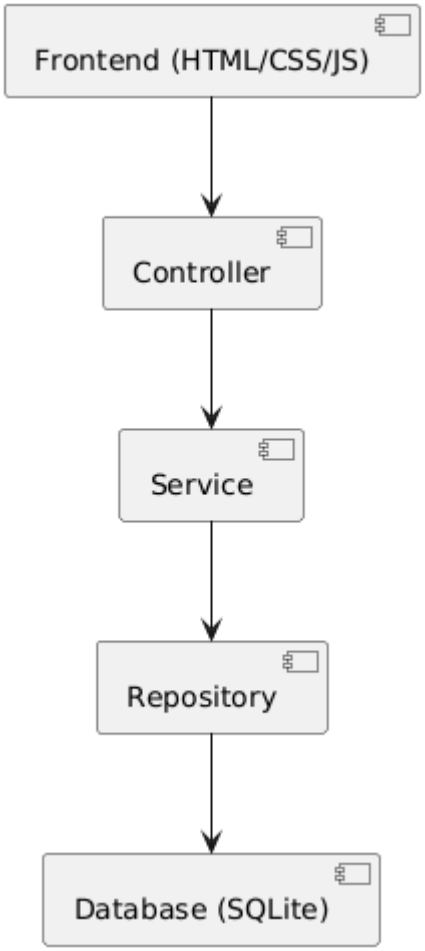


5. VISÃO LÓGICA

Camadas do sistema

1. Apresentação (View)
2. Controller (Flask)
3. Service (Regras de negócio)
4. Model (Domínio)
5. Repository (Persistência)

DIAGRAMA DE CAMADAS



6. VISÃO DE IMPLEMENTAÇÃO

Caso de uso: Login

Componentes

- Usuario (Model)
- AuthController
- AuthService
- UsuarioRepository

DIAGRAMA DE CLASSES

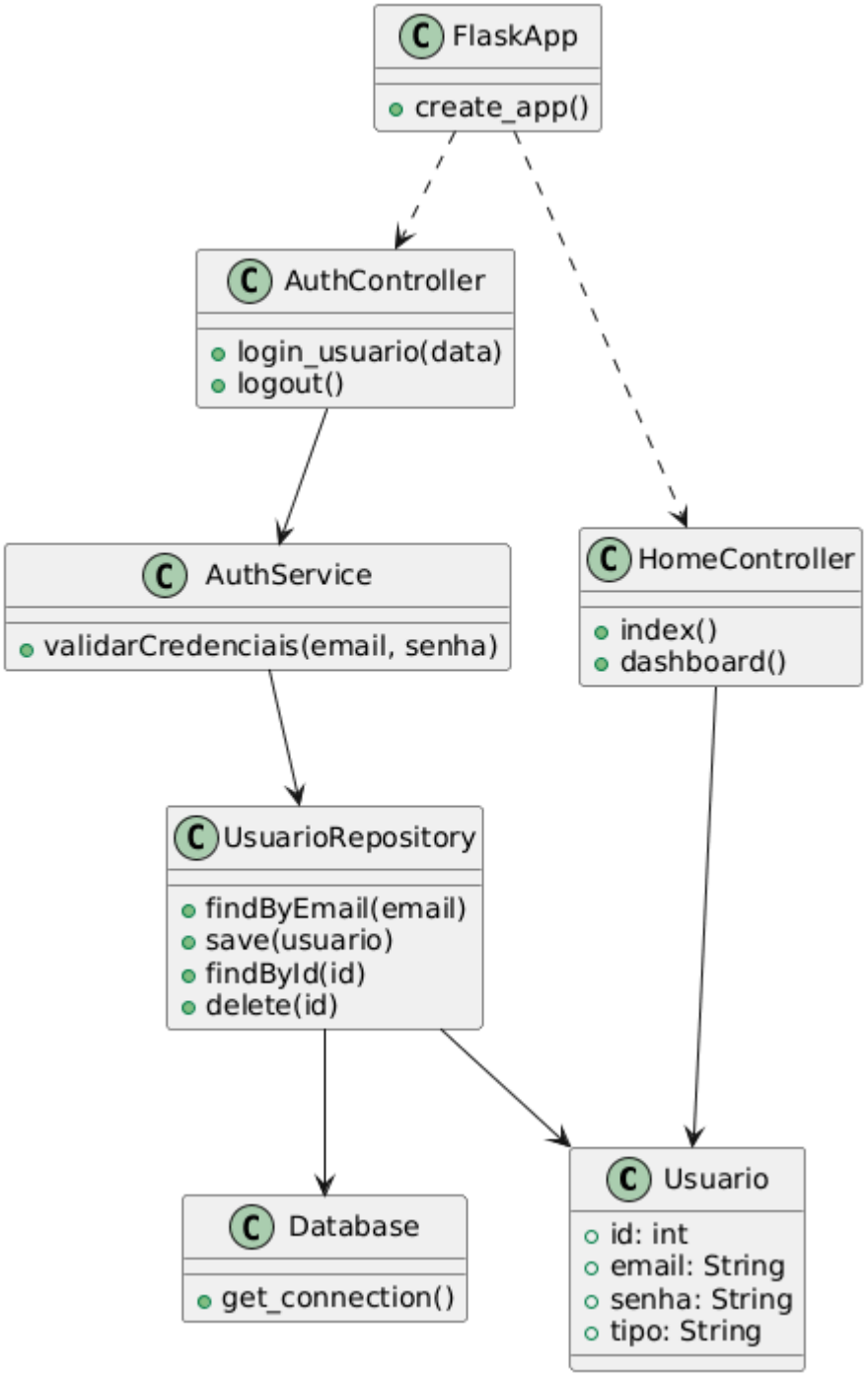
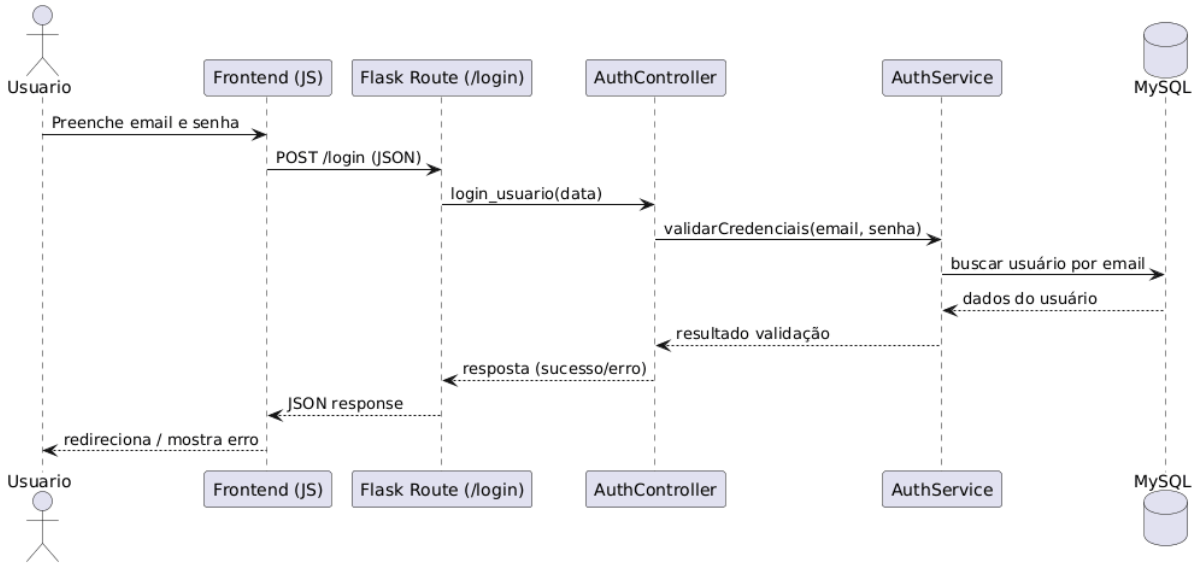


DIAGRAMA DE SEQUÊNCIA

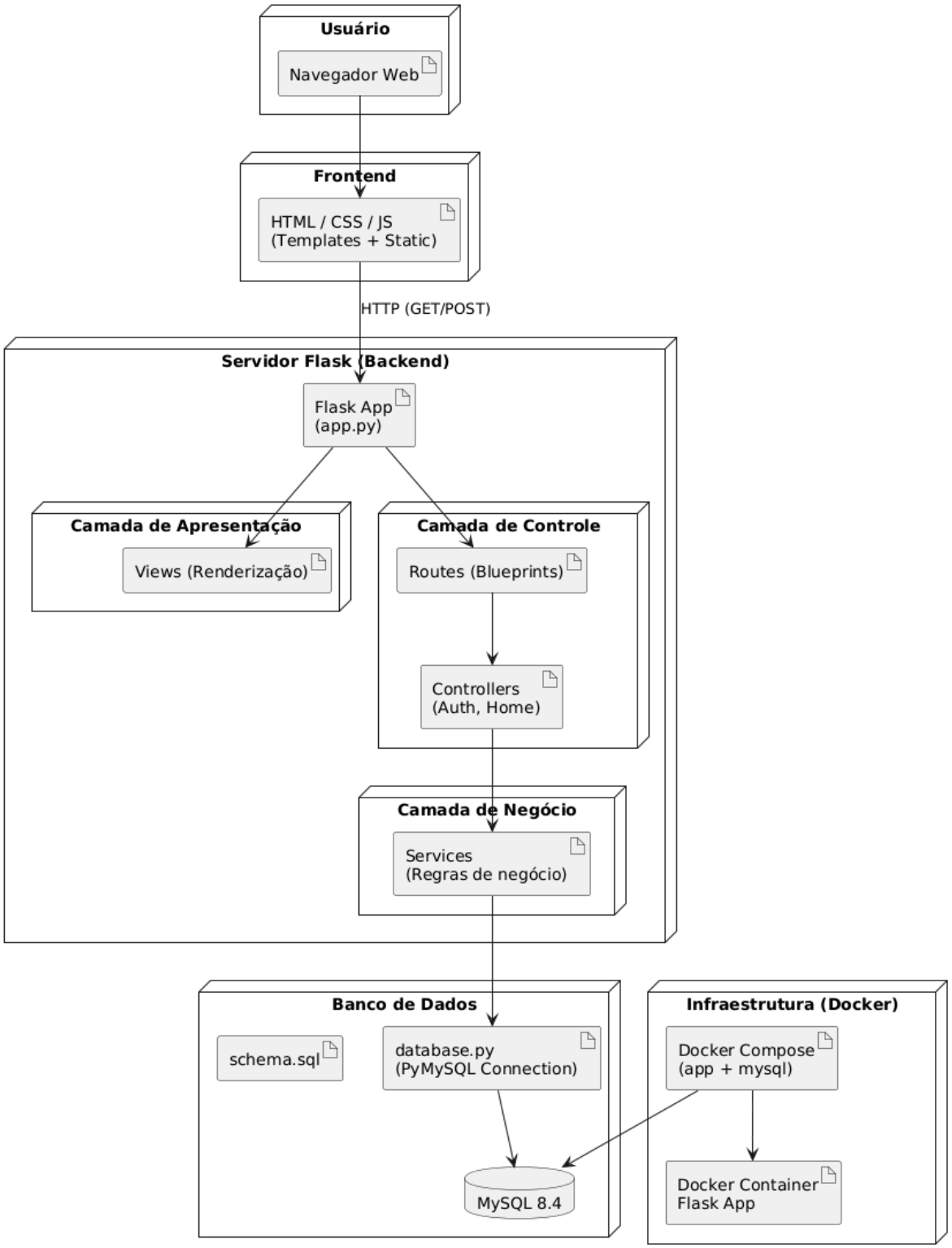


7. VISÃO DE IMPLANTAÇÃO

Estrutura física

- Cliente: navegador
- Servidor: Flask
- Banco: SQLite

DIAGRAMA DE IMPLANTAÇÃO



8. DIMENSIONAMENTO E PERFORMANCE

- Usuários simultâneos: 10–50
- Requisições: 200–500/dia
- Resposta média: < 2s

9. QUALIDADE DO SISTEMA

Atributo	Estratégia adotada
Escalabilidade	Arquitetura em camadas
Segurança	Autenticação e hash de senha
Confiabilidade	Tratamento de erros e validações
Manutenibilidade	Separação de responsabilidades + mvc
Portabilidade	Aplicação web
Desempenho	Consultas otimizadas e uso de ORM para acesso eficiente ao banco
Usabilidade	Interface web simples e intuitiva para usuários da secretaria
Integridade dos dados	Uso de banco relacional + constraints (chaves primárias e únicas)

10. JUSTIFICATIVA ARQUITETURAL

A arquitetura em camadas e o padrão MVC foram escolhidos para garantir separação de responsabilidades, facilitando manutenção e evolução do sistema.

O Flask foi adotado por sua leveza e flexibilidade.

O SQLAlchemy reduz acoplamento com o banco de dados.

O sistema utiliza SQLite no ambiente de desenvolvimento, podendo evoluir para PostgreSQL em produção.